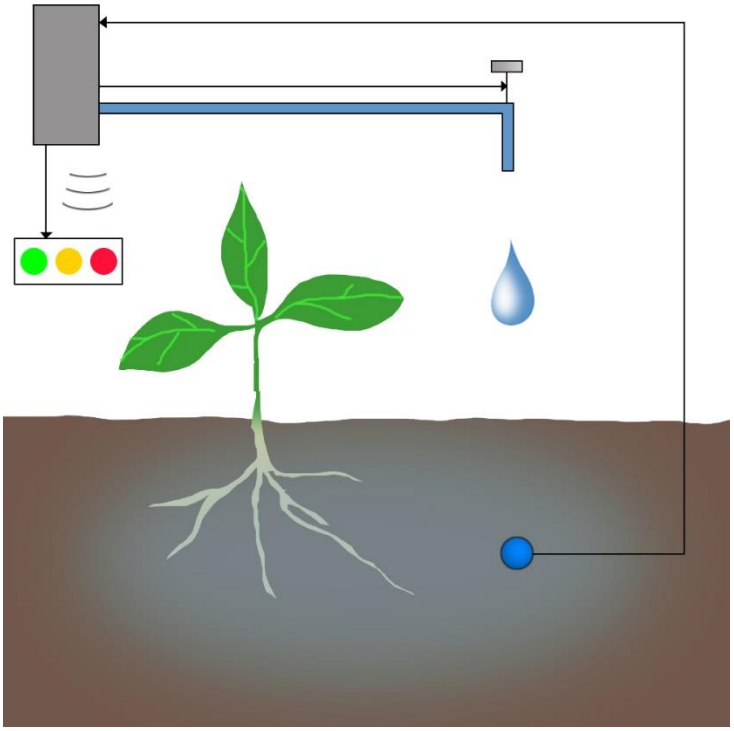
Juha Laurila

Älykkäät järjestelmät -projekti:

Kastelujärjestelmän automatisointi



Insinööri

Tieto- ja viestintätekniikka

Kevät 2024

KAMK • University of Applied Sciences

Sisällys

1 Johdanto ja projektin määrittely 1

2 Ajankäyttösuunnitelma 2

3 Kastelujärjestelmä ja sen automatisointi 3

3.1 Olemassa olevan kastelujärjestelmän toteutus 3
3.2 Järjestelmän automatisointisuunnitelma 4

4 Projektissa tarvittavat komponentit 5

4.1 Arduino Uno R3 5
4.2 Anturit 6
4.2.1 Resistiivinen kosteusanturi HW-103 6
4.2.2 Kapasitiivinen kosteusanturi HW-390 7
4.2.3 Etäisyysanturi HC-SRO4 8
4.3 Moottorinohjausmoduuli L298N 9
4.4 Moottorit 10
4.5 Kytkentä- ja ilmaisinkomponentit 10
4.6 Bluetooth-moduuli Adafruit nRF 8001 10

5 Toteutus 12

5.1 Anturien testaus ja käyttöönotto 12
5.2 Bluetooth-moduulin käyttöönotto 12
5.3 Moottorien käyttöönotto 13
5.4 Kastelujärjestelmädemon kytkentä 14
5.5 Ohjelman toiminta sanallisesti ja vuokaaviona (ohjelman logiikka) 16
5.6 Ohjelmakoodi 19

6 Yhteenveto 29

1 Johdanto ja projektin määrittely

Projektityön tavoitteena oli suunnitella ja toteuttaa jo olemassa olevalle, mutta keskeneräiselle kasvien kastelujärjestelmälle automatisoituja toimintoja. Automatiikan, sen seurannan ja järjestelmän tilan indikoinnin avulla kastelujärjestelmä toimisi itsekseen ja saataisiin lisäksi reaaliaikaista tietoa kastelujärjestelmän tilasta.

Projektiin kuului kasvien vedensaannin ja sopivan kastelun varmistaminen, järjestelmän tilan ilmaiseminen sekä tarpeelliset hälytykset, kiertovesipumpun ja kastelupumpun ohjaus ja niiden automaattinen sammuttaminen veden loppuessa sekä etävalvontaominaisuudet.

Hälytykset ja informaatio järjestelmän tilasta osoitettaisiin paikallisesti ledeillä, äänimerkeillä sekä Bluetoothin välityksellä esim. puhelimeen.

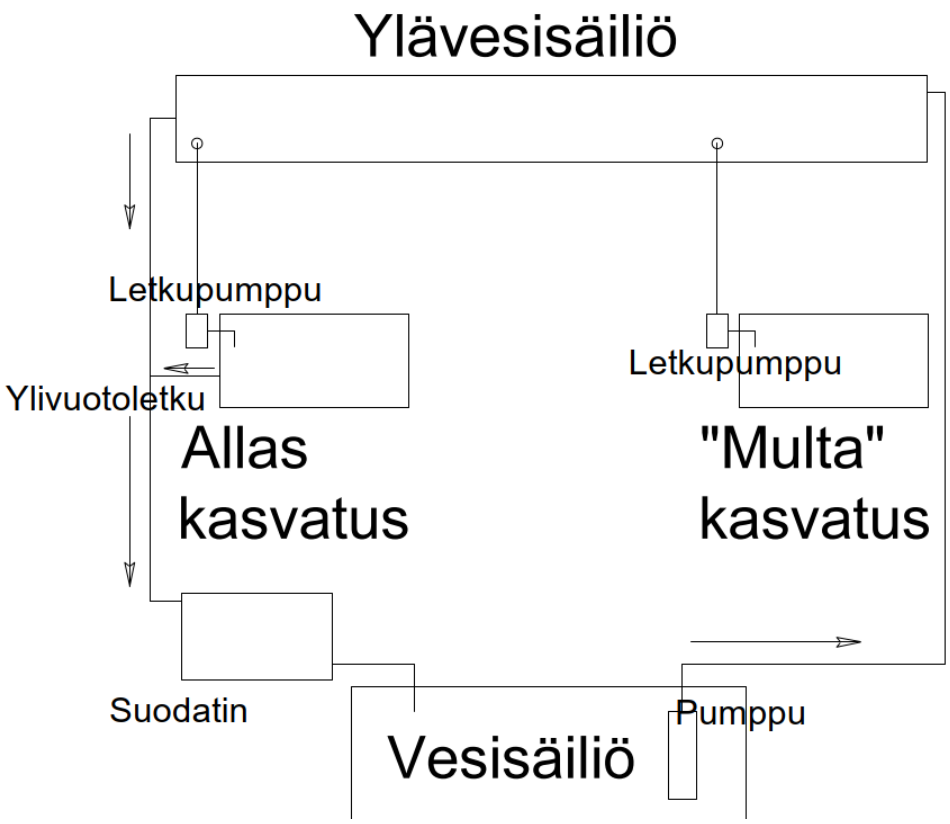
2 Ajankäyttösuunnitelma

Ajankäytöllisesti projektissa oli neljä osiota. Ensimmäiseen kuului olemassa olevaan järjestelmään perehtyminen ja automatisointisuunnitelman laatiminen. Toiseen osioon kuului tarvittaviin komponentteihin sekä niiden ominaisuuksiin tutustuminen sekä teoriassa, että käytännössä testikytkentöjen ja -ohjelmien avulla. Kolmas osio oli lopullisen ohjelman ja vastaavan kytkennän toteuttaminen toimivaksi demokokonaisuudeksi. Neljäs osio oli kaiken dokumentointi kirjalliseen raporttiin. Viides osio olisi ollut järjestelmän liittäminen osaksi oikeaa kastelujärjestelmää, toimivuuden seuranta, tulosten ylös kirjaaminen ja mahdollinen fyysisen järjestelmän, komponenttien ja ohjelmallinen hienosäätö toiminnan parantamiseksi. Viidennen osion jätin kuitenkin johtuen aikaresurssien vähyydestä tämän projektin ulkopuolelle.

3. Kastelujärjestelmä ja sen automatisointi

3.1. Olemassa olevan kastelujärjestelmän toteutus

Jo osittain rakennettu kastelujärjestelmädemo sijaitsee Kajaanin Ammattikorkeakoulun tiloissa, tarkasti ottaen fysiikan laboratorion sisäänkäynnin vieressä olevassa lukittavassa ja valaistussa kaapissa. Kaapin sisällä oleva kastelujärjestelmä rakentuu alimpana olevasta vesisäiliöstä, josta vesi pumpataan kaapin yläosassa sijaitsevaan paksusta muoviputkesta rakennettuun ylävesisäiliöön, pitäen sen sisällä aina riittävästi kiertävää vettä. Ylävesisäiliössä on yhteet, joista lähtee venttiileillä/letkupumpuilla varustetut kasteluvesiputket kasvatuslaatikoihin. Ylimääräinen kiertävä vesi poistuu ylävesisäiliöstä paluuputkea pitkin suodatettuna takaisin alimpana olevaan nostopumpulla varustettuun vesiastiaan.



Kuva 1. Olemassa olevan kastelujärjestelmän periaatekuva.

3.2. Järjestelmän automatisointisuunnitelma

Ilman automaattista ohjausta olemassa oleva toteutus ei luonnollisestikaan toimi ilman jatkuvaa henkilön seurantaa. Projektin päämäärä olikin suunnitella järjestelmälle toiminta- ja valvonta-automatiikka, jotta kastelujärjestelmä voisi toimia pitempiä aikoja luotettavasti myös itsekseen.

Perusasia veden saannille on luotettavasti toimiva kiertovesijärjestelmä. Alinna olevan vesiastian veden määrää pitäisi myös voida tarkkailla ilman säiliöön kurkistamista. Indikaatiot ja hälytykset tyhjenevästä astiasta pitäisi toteuttaa ensinnäkin paikallisesti kaapin ulkopuolella olevin ledein ja äänimerkein. Veden tason edelleen vähentyessä tulisi pumppuhälytys – ennakkovaroitus kohta pysähtyvästä pumpusta, jos vettä ei lisätä pian säiliöön. Jos vesi vähentyisi edelleen, tapahtuisi pumpun pysähtyminen kuitenkin vielä siinä vaiheessa, kun pumpulle olisi vielä hieman vettä – pumpun tyhjänä pyörimisen estämiseksi.

Jotta mullan kastelu toimisi automaattisesti, pitäisi kasvatuslaatikoissa olla luonnollisesti kosteusanturit, jotka seuraavat koko ajan mullan kosteutta. Kosteusanturien antaman tiedon mukaan ohjattaisiin sitten ylävesisäiliöstä tulevien kasteluputkien venttiileitä päästämään tarpeen mukaan lisää kasteluvettä multaan, pitäen mullan kosteusolosuhteet kyseiselle kasvatettavalle kasville optimaalisena.

Paluuputkeen on asennettu suodatin, jonka tilaa pitäisi pystyä myös tarkkailemaan. Suodattimen tehtävä on puhdistaa kiertovedestä kaikki ylimääräinen roska ja orgaaninen aines kuten vedessä mahdollisesti kasvava levä. Suodattimen tukkeutumisen aste voitaisiin todeta esim. paineeroanturilla, joka antaisi lopulta hälytyksen suodattimen puhdistus- tai vaihtotarpeesta.

Paikallisesti toimivan tilanilmaisun lisäksi olisi hyvä, jos järjestelmää voitaisiin seurata etänä, joko Bluetoothin tai Wifin välityksellä. Ensiksi ajattelin tehdä projektin, jossa käytettäisi internet-pohjaista seurantarajapintaa pilvestä käsin, esim. sitä varten tehdyllä mobiilisovelluksella. Ajan vähyyden vuoksi päätin toteuttaa etäseurannan kuitenkin BLE:n avulla, Arduinoon liitettävää Adafruit nRF 8100-moduulia käyttäen, jolloin etävalvontaetäisyys voisi olla maksimissaan n. 10 m:n luokkaa.

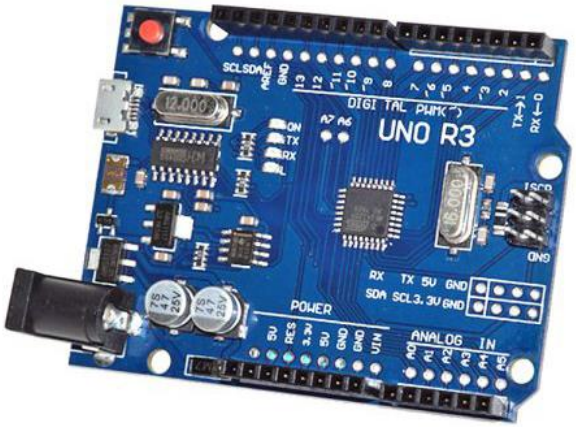
4 Projektissa tarvittavat komponentit

4.1 Arduino Uno R3

Käytettävissä oli Arduino UNO R3 -kehitysalusta, joka on 5 V:n käyttöjännitteellä toimiva ATmega328P:een perustuva mikro-ohjainkortti. Siinä on 14 digitaalista tulo-/lähtönastaa, joista 6:ta voidaan käyttää PWM-lähtöinä (Pulse Width Modulation), 6 analogista tuloa, 16 MHz keraaminen resonaattori, USB-liitäntä, virtaliitin, ICSP-kanta (in-circuit serial programming, piirin sisäinen sarjaohjelmointi) ja nollauspainike [1].

ATmega328P:ssä on myös 1 kb EEPROM, muisti, joka ei tyhjene, kun virta katkaistaan. Arduino-ohjainkorttia ohjelmoidaan Arduino IDE -ohjelmointiympäristöllä. Arduino IDE:ssä käytetään omaa C++:een perustuvaa ohjelmointikieltä, joka myös käännetään ja ajetaan Arduino mikro-ohjaimelle kyseisestä ympäristöstä USB-kaapelia käyttäen [1].

Arduino Uno R3-kortilla ei ole sisäänrakennettua BLE- eikä WiFi-ominaisuutta. BLE-ominaisuus otetaan käyttöön Unoon liitettävällä erillisellä Adafruit nRF8001 -moduulilla.



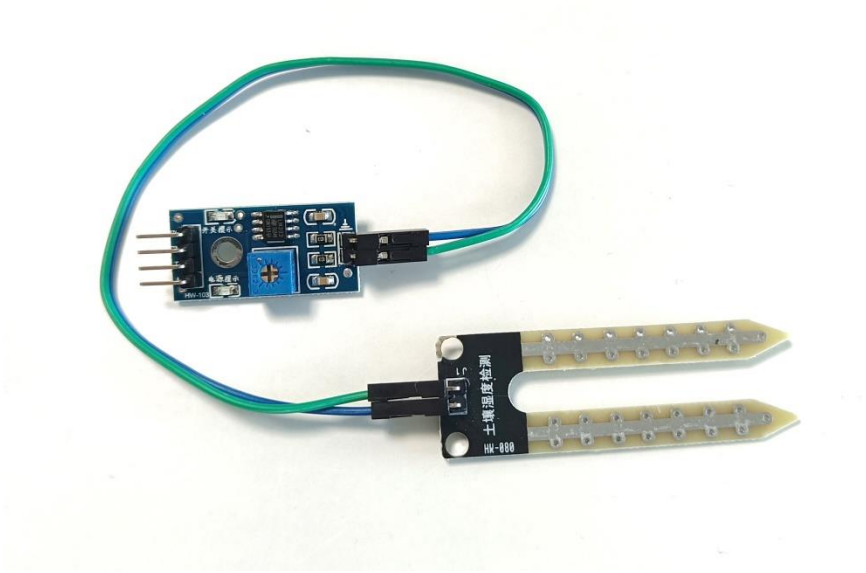
Kuva 2. Arduino Uno R3 kehitysalusta (mikro-USB:lla varustettu).

4.2 Anturit

Projektia aloittaessani sain koululta kolme erityyppistä anturia projektiin. Silloin ei ollut vielä tiedossa, mitä projektissa tulisi käyttää, mutta se oli selvitettävä eli kaikkia olisi testattava. Varsinaisia kosteudenmittausantureita oli kahta tyyppiä, resistanssin muutokseen perustuva HW-103 sekä kapasitanssin muutokseen perustuva HW-309. Lisäksi mukaan tuli ultraääneen perustuva etäisyysanturi HC-SRO4. Anturien lähtökohtainen käyttötarkoitus projektissa oli kasteluvesiastian nestepinnan korkeuden mittaus ja projektin edetessä ajattelin ottaa mukaan myös itse kasvualustan kosteuden mittauksen optimaalisen kastelun toteuttamista varten.

4.2.1 Resistiivinen kosteusanturi HW-103

HW-103 on resistiivinen kosteusanturi, jonka haarukkamaisessa, esim. maaperään työnnettävässä mittapäässä on kaksi paljasta metallielektrodia. Anturi toimii ikään kuin potentiometrinä, maaperän kosteuden mukaan muuttuvana vastuksena. Mittaustieto perustuu maaperän johtavuuden muutokseen kosteuden mukaan. Kun vettä on maaperässä runsaammin, on sen virran johtavuus parempi ja resistanssi elektrodien välillä pienempi ja mitä vähemmän vettä on maaperässä, sitä pienempi on vastaavasti sen johtavuus ja suurempi resistanssi. HW-103 anturi tuottaa tämän resistanssin mukaisen lähtöjännitteen, jonka avulla voidaan määrittää maaperän kosteustaso [2].



Kuva 3. Resistiivinen kosteusanturi HW-103.

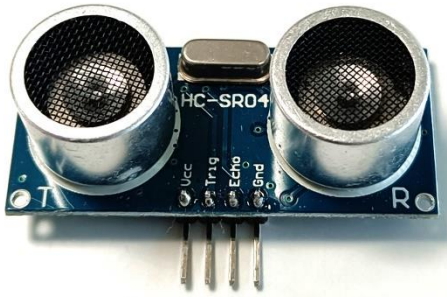
4.2.2 Kapasitiivinen kosteusanturi HW-390

HW-390 on kapasitiivinen kosteusanturi, jonka mittaus perustuu anturin kapasitanssin muutokseen ympäröivän kosteuden mukaan. Tässäkin anturissa on maaperään työnnettävä mittapääosuus, mutta sen sähköiset osat ovat eristetty suoralta kosketukselta, toisin sanoen paljaita metalliosia ei ole, eikä niiden välillä edes kulje virtaa. HW-390 sisältää kanttiaaltoa tuottavan oskillaattorin. Kun anturin ympäristön kapasitanssi muuttuu, muuttuu myös oskillaattorin lähettämän kanttiaallon taajuus. Tämä taajuuden muutos muokataan anturin piirillä vaikuttamaan anturin analogialähdössä olevan lähtöjännitteen tasoon, jonka perusteella kosteustaso voidaan määrittää [2][3].

Kuva 4. Kapasitiivinen kosteusanturi HW-390.

4.2.3 Etäisyysanturi HC-SRO4

HC-SRO4:lla on etäisyysanturi, jonka toiminta perustuu ultraäänen lähetyksen ja vastaanottamisen välillä kuluvan ajan mittaukseen. Anturissa on ultraäänen lähetinosa ja vastaanotinosa, joka ottaa esteestä takaisin heijastuneen ultraäänen vastaan. Mitä lähempänä este on, sitä pienempi lähetyksen ja vastaanottamisen aikaero ja mitä kauempana este, on luonnollisesti aikaero suurempi. Aikaeron muutokset vaikuttavat sitten HC-SRO4-anturin lähtöjännitetasoon, josta ultraäänen heijastumisen aiheuttavan esteen etäisyys voidaan ohjelmallisesti laskea [4].



Kuva 5. Etäisyysanturi HC-SRO4.

4.3 Moottorin ohjausmoduuli L298N

Kastelujärjestelmän pohjalla olevassa vesiastiassa on kiertovesipumppu ja kastelujärjestelmässä on kastelupumppu, joiden toimintaa täytyy myös pystyä hallitsemaan Arduinolla. Pumput pitää myös voida pysäyttää automaattisesti silloin, kun vesi loppuu, sillä pumpun tyhjänä pyöriminen vaurioittaa pumppua.

Arduinolla voidaan ohjata suoraan vain heikkotehoisia komponentteja, kuten LEDejä ja niitäkin sarjakytkettyjen vastuksien kanssa, Arduinon ja LEDien suojaamiseksi liian korkealta virralta. Koska moottori tarvitsee pyöriäkseen LEDejä reilusti suuremman tehon (P=UI), kuin mitä Arduino pystyy suoraan antamaan, on pumppumoottoria ohjattava epäsuorasti erityisen mootto-

rinohjaimen avulla. Arduino ohjaa suoraan vain moottorinohjausmoduulia pienitehoisella signaalilla, joka sitten kytkee varsinaisen moottorille menevän erillisen sähkönsyötön päälle ja pois.

L298N on DC- ja askelmoottorien ohjaukseen suunniteltu suuren tehon moduuli, joka mahdollistaa maksimissaan neljän moottorin ohjauksen sekä kahden moottorin ohjauksen suunnan ja nopeuden säädöllä. Nopeuden säätö on toteutettu PWM-tekniikalla (Pulse Width Modulation, pulssinleveysmodulaatio). Sen avulla säädetään moottorille menevän jännitteen keskiarvoa kytkemällä Arduinon PWM-signaalilla nopeasti virta päälle ja pois, jolloin keskimääräinen moottorille menevä jännite riippuu signaalin High ja Low-tilojen kokonaisaikojen välisestä suhteesta yhden toimintajakson aikana [5].

Moottorin pyörimissuunta vaihdetaan moottorin virtajohtojen napaisuutta muuttamalla. Käytännössä pyörimissuunnan vaihto toteutetaan L298N:ssäkin, kuten monissa muissakin vastaavissa moduuleissa H-Bridgellä (H-silta). Moottorinohjausmoduulissa kaksi H-siltaa, joissa kummassakin on neljä kytkinelementtiä, jotka ajettavan moottorin kanssa muodostavat ikään kuin H:n muotoinen virtapiirin. Kytkimen tiloja vaihtamalla saadaan virran suunta ja sitä kautta moottorin pyörimissuunta käännettyä [5].

Kuva 6. Moottorinohjausmoduuli L298N.

4.4 Moottorit

Sain koululta mukaani moottorin ohjausmoduuli L298N:n lisäksi pienen vaihteella varustetun DC-moottorin sekä suuremman 12 V DC-moottorilla varustetun uppopumpun. Käytin pienempää moottoria kastelupumpun moottorina ja suurempaa uppopumppua kiertovesipumppuna. Molempia pystyi ohjaamaan moottorinohjausmoduuli L298N:lla yhtä aikaa. Virtalähteenä moottoreille käytin kastelujärjestelmädemossa 9 V:n paristoa. Moottoreille olisi saanut ohjausmoduulin puolesta toteutettua pyörimissuunnan vaihdon ja nopeuden säädön, mutta Arduinossa loppuivat ensinnäkin pinnit ominaisuuden johdottamiselle ja toisekseen, kastelujärjestelmässä ei ainakaan demoversiossa tarvinnut näitä ominaisuuksia. Mikäli projektina olisi ollut esim. jonkinlainen robottiauto olisivat ne olleet välttämättömiä, sillä ilman näitä kahta ominaisuutta auto kulkisi vain yhteen suuntaan ja tasanopeudella.

4.5 Kytkentä- ja ilmaisinkomponentit

Tällaisessa kehitysprojektissa tarvitaan luonnollisesti erilaisia kytkentä- ja ilmaisinkomponentteja testaukseen ja toiminnallisuuksien toteuttamiseen. Testikytkennät ovat kätevintä tehdä käyttäen kytkentäalustaa, johon saadaan nopeasti rakennettua ja purettua kytkentöjä erimittaisia kytkentälankoja käyttäen tarvitsematta juottaa mitään. Paikalliseen järjestelmän tilan ilmaisimeen tarvitaan erivärisiä LED-valoja. LED (Light emitting diode) on puolijohdekomponentti, joka lähettää valoa , kun virta kulkee sen läpi. LEDit tarvitesevat etuvastuksen (esim. 220 ohm virran rajoittamiseen), jotta LED ja Arduino kestävät kuormituksen. Ääni-ilmaisuun tarvitaan tietysti pienitehoinen kaiutin (riittävän suurella impedanssilla, esim. 8 ohm), jotta Arduino ei ylikuormittuisi. Kaiuttimen ja GND:n välille on hyvä laittaa lisäksi esim. 220 ohmin vastus, joka suojaa Arduinoa ja kaiutinta liian suurelta virralta. Tarvittaessa järjestelmän tilojen tarkasteluun / valintaan paikallisesti voitaisiin käyttää painonappeja.

4.6 Bluetooth LE -moduuli Adafruit nRF 8001

nRF8001 BLE-moduuli (BLE = Bluetooth Low Energy) mahdollistaa helppokäyttöisen langattoman yhteyden muodostamisen Arduinon ja yhteensopivan iOS- tai Android-laitteen välillä. nRF8001-moduulia voidaan käyttää turvallisesti 5 V:n logiikan kanssa, ja se käyttää mukautettua SPI-tyyppistä väylää Arduinon kanssa kommunikointiin. Se toimii simuloimalla UART-laitetta ja lähettämällä ASCII-muotoista tietoa edestakaisin laitteiden välillä. Käyttäjä voi päättää mitä tietoja lähetetään ja mitä niillä tehdään yhteyden kommassakin päässä.

nRF8001 kytketään Arduinoon johdottamalla ja otetaan ohjelmallisesti käyttöön UART-kirjaston avulla. Arduinoon kytketyn nRF 8001-moduulin ja Android-laitteen välinen langaton yhteydenpito onnistuu n. 10 metriin saakka [6].

SPI (Serial Peripheral Interface) on synkronisen sarjaviestinnän *tosiasiallinen* standardi (monilla muunnelmilla), jota käytetään pääasiassa sulautetuissa järjestelmissä integroitujen piirien väliseen lyhyen matkan langalliseen viestintään [7].

UART (Universal Asynchronous Receiver Transmitter) eli sarjaliikennepiiri on laitteisto tai mikropiiri, joka muuntaa rinnakkaismuotoista tietoa sarjamuotoiseksi ja päinvastoin. UART-piirille syötetään tavu, ja se pilkkoo sen biteiksi ja lähettää sarjaliitännän yli bitti kerrallaan ohjelmoidulla nopeudella ja muilla protokollan vaatimuksilla, esim. ajoituksesta ja tarkistussummasta. Kehittyneemmissä versioissa on muutaman tavun puskurimuisti. UART-piiriä käytetään asynkronisessa sarjamuotoisessa tietoliikenteessä tietokoneen ja oheislaitteen välillä [8].

Kuva 7. Bluetooth BLE-moduuli nRF8001.

5 Toteutus

5.1 Anturien käyttöönotto ja testaus

Ensimmäinen toimenpide ryhtyessäni toteuttaman projektia oli antureiden käyttöönotto ja testaus. Kaikki saamani kolme anturia - resistiivinen kosteusanturi, kapasitiivinen kosteusanturi sekä etäisyysanturi olivat minulle entuudestaan tuntemattomia, joten liikkeelle oli lähdettävä niiden toiminnan, mallikoodien ja kytkentälogiikoiden selvittämisestä. Kaikista löytyi onneksi aika helposti valmiita malliesimerkkejä niin kytkennöistä kuin ohjelmista. Pääsin nopeasti myös testaamaan niitä ensin pelkästään sarjamonitorista, mutta heti perään myös tekemällä ohjelmia ja kytkentöjä, jossa eri kosteusarvojen mukaan syttyivät niitä vastaavat ledit. Kosteusantureita testasin mm. kostealla pyyhkeellä, upottamalla antureita osittain veteen sekä kapasitiivista kosteusanturia lisäksi vedellä täytetyn astian kyljessä. Etäisyysanturia testasin niin ikään samalla periaatteella, tehden ehtolauseita eri etäisyyksille ja niitä vastaavat ledit aina syttymään. Testasin sitten etäisyysanturia mittaamaan myös veden pinnan etäisyyttä ylhäältä päin, mikä kyllä toimi ihan luotettavasti.

Kosteusanturiksi projektiin valikoitui kuitenkin HW-390 kapasitiivinen kosteusanturi. Se on monikäyttöinen ja kestävä maaperän kosteuden mittausanturi, jossa ei ole elektrolyysin takia hapettuvia paljaita kontaktipintoja, toisin kuin resistanssin muutosta mittaavissa kosteusantureissa. Niinpä se on pitkäikäisempi esim. kosteaan multaan sijoitettuna. Anturin toiminta perustuu kapasitanssin muutokseen maaperän kosteuden vaihteluiden mukaan, mitä suurempi kosteus, sitä suurempi kapasitanssi. HW-390 toimii myös maaperän kosteusmittauksen lisäksi esim. muovisen vesiastian ulkokyljessä. Näin se soveltuu huomattavasti paremmin myös nestetason mittaukseen verrattuna resistiiviseen anturiin, jossa pitää olla suora kontakti nesteeseen tai etäisyysanturiin, jossa pitää olla suora yhteys nestepintaan. Näin anturi saadaan mahdollisimman mielekkääseen ja suojattuun paikkaan myös todellisessa järjestelmässä.

5.2 Bluetooth-moduulin kytkentä ja käyttöönotto

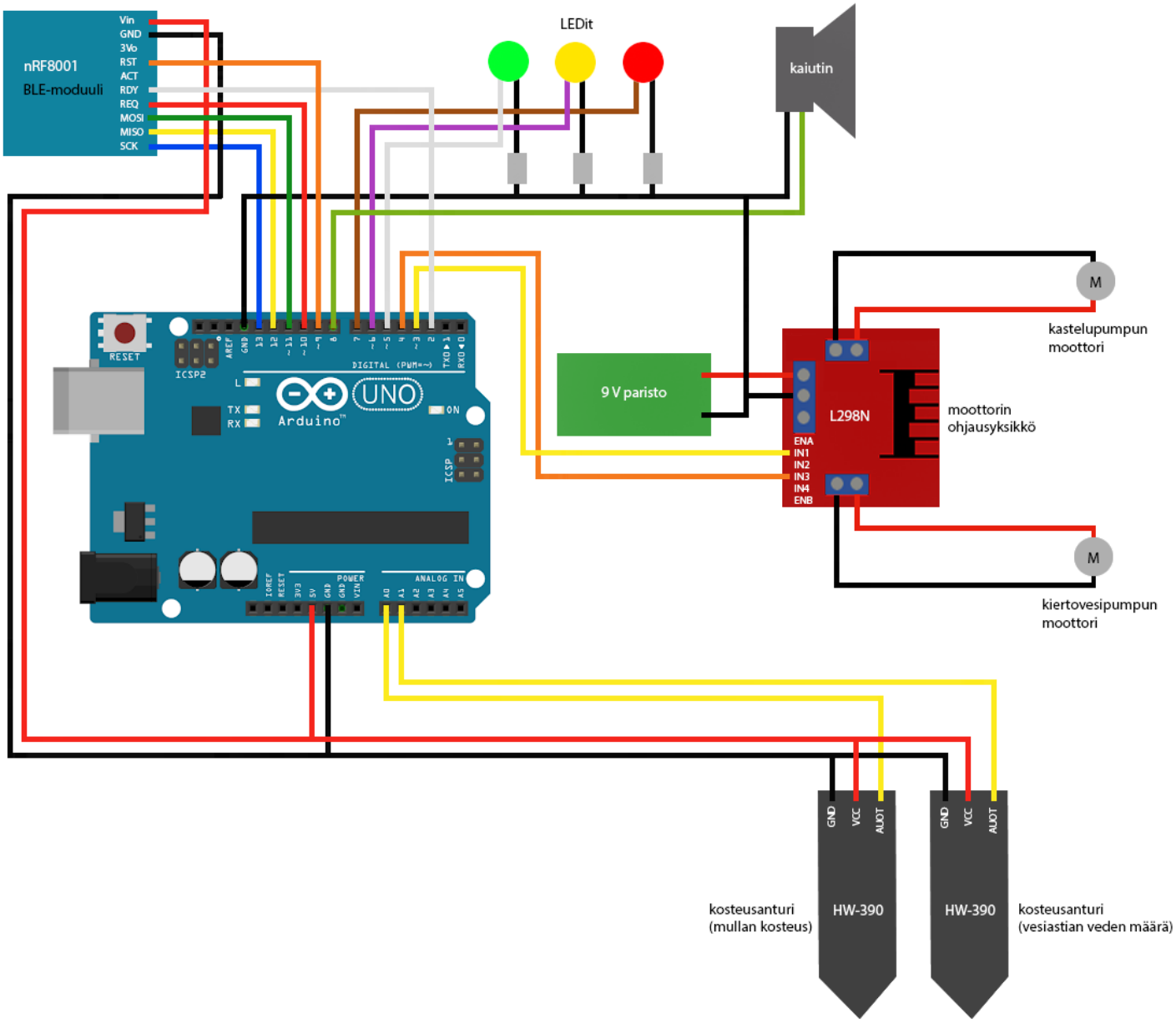
Löysin internetistä useita ohjeita, miten BLE-moduuli Adafuit nRF8001 kytketään Arduinoon ja otetaan käyttöön ohjelmallisesti. Tein ohjeen mukaisen kytkennän ja testasin parilla malliohjelmalla toimivuutta. Tämän jälkeen olin valmis siirtämään BLE-toiminnallisuuden osaksi varsinaista projektin koodia.

- VIN liitetään Arduino 5V nastaan
- GND liitetään Arduino-maahan
- SCK kytkeytyy SPI-kelloon.
- MISO muodostaa yhteyden SPI MISO -palveluun (Master In Slave Out)
- MOSI muodostaa yhteyden SPI MOSI:iin (Master Out Slave In)
- REQ muodostaa yhteyden SPI Chip Select -nastaan.
- RST muodostaa yhteyden Digital 9:ään - tämä on tarkoitettu kortin nollaamiseen käynnistyksen yhteydessä
- RDY on keskeytys nRF8001:stä

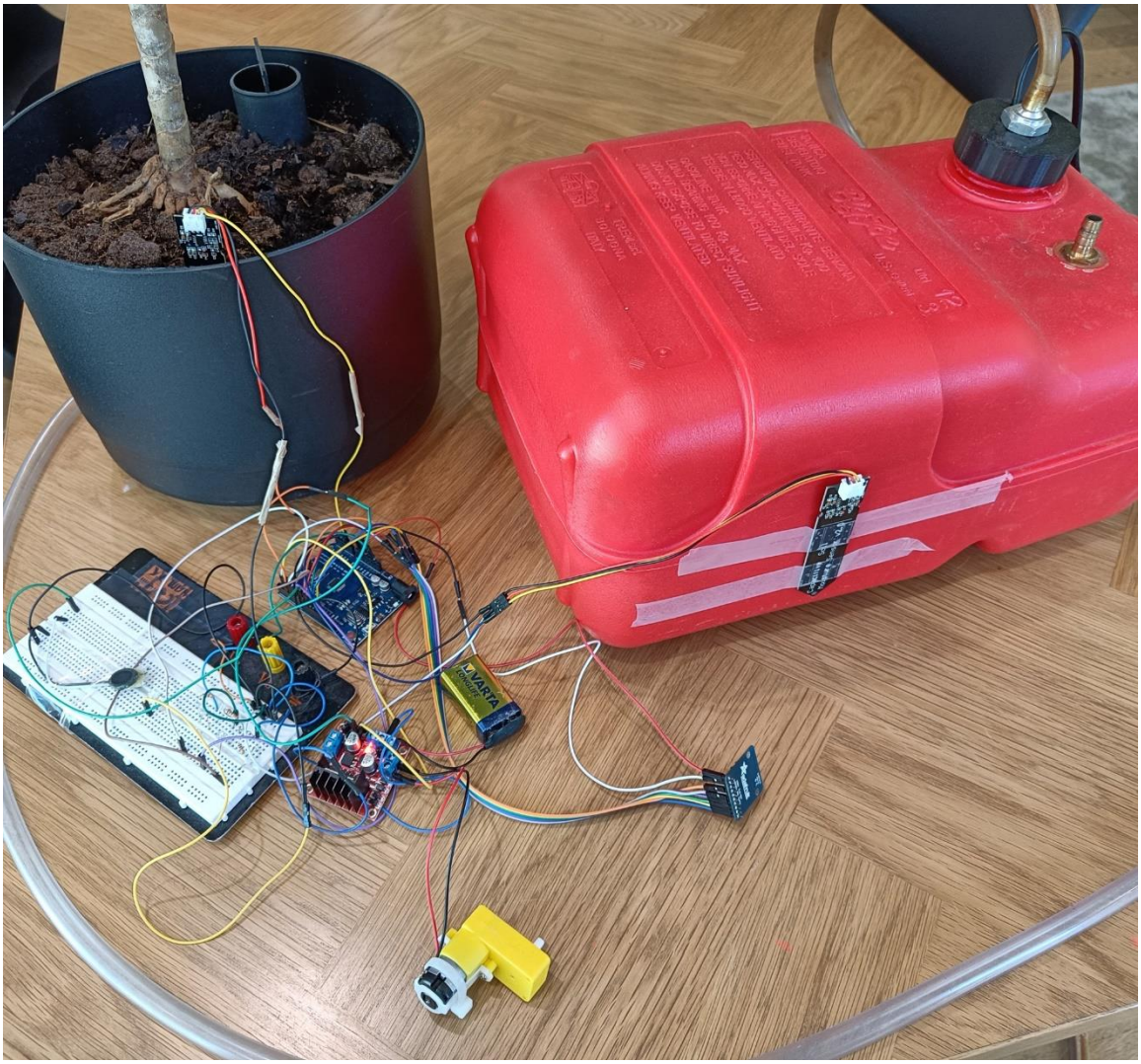
5.3 Moottorien käyttöönotto

Moottorien ohjausmoduulista löytyi myös kattavasti tietoa ja toteutettuja esimerkkejä. Moottorien käyttöönotto onnistuikin nopeasti ohjeita tarkasti seuraamalla ja tekemällä johdotukset ja ohjelma esimerkkien mukaan. Pienempi moottoreista piti käydessään huomattavaa ääntä, arvelin äänen tulevan vaihteesta. Suurempi uppopumpun moottori sen sijaan oli erittäin hiljainen käyntiääneltään. Otin ensimmäisenä käyttöön pienemmän moottorin kiertovesipumpun toimintaan. Laajensin sitten ohjelmaa kastelutoiminnallisuudella, jota indikoin aluksi ylimääräisellä ledillä. Vaikka suunnitteli aluksi jättäväni ledin kastelun merkiksi varsinaisen moottorin käyttöönoton lisäksi, muutin ledin lähdön uppopumpun lähdöksi, koska yhtään pinniä ei ollut enää vapaana. Lopuksi vaihdoin kiertovesipumpuksi uppopumpun ja kastelupumpuksi pienemmän moottorin.

5.4. Kastelujärjestelmädemon kytkentä



Kuva 7. Kosteudenhallintajärjestelmän automatisoinnin kytkentäperiaate.



Kuva 8. Kosteudenhallintajärjestelmän automatisoinnin demonstraatio.

5.5 Ohjelman toiminta sanallisesti ja vuokaaviona (ohjelman logiikka)

Ohjelma on suunniteltu Arduino Uno -kehitysalustalla ajettavaksi. Ohjelma rakentuu muutamasta perustoiminnosta, joilla toteutetaan kastelujärjestelmän automatisoitu tilavalvonta. Oleellisimpia näistä ovat kosteusanturien tietojen käsittely, erilaiset indikaatiot ja hälytykset järjestelmän tilasta, kiertovesipumpun ja kastelupumpun ohjaukseen tarvittava toiminnallisuus sekä etävalvonnan toteuttaminen käytettäväksi puhelimella tai muussa etävalvontapisteessä Bluetoothin avulla.

Kun ohjelma käynnistyy, importoidaan tarvittavat kirjastot toimintojen toteuttamiseksi. Näitä ovat ensinnäkin *otsikkotiedosto ”pitch.h”* eri korkuisten äänien tuottamiseksi hälytyksissä. Lisäksi tarvitaan *”spi.h”* -kirjastoa (serial peripheral interface) lyhyen matkan langalliseen viestintään Arduino masterlaitteen ja siihen kytkettävän bluetooth slave-laitteen (Adalfruit nRF8001) välillä. Laitteen etäkäyttämiseksi bluetoothilla esim. puhelimella, tarvitaan vielä kirjasto *”Adalfruit_BLE_UART.h”* Bluetooth-toiminnallisuuden toteuttamiseksi.

Seuraavaksi tehdään määritykset Adafruit nRF8001 -bluetooth-moduulin ottamiseksi käyttöön. Aluksi määritetään Arduino Uno-kortin nastat ulkoisen bluetooth-moduulin käyttämiseksi. BLE_REQ on nRF8001:n piirinvalintanasta, BLE_RDY on datan valmiusnasta (keskeytyslähtö katkeamisesta mikrokontrolleriin, joka ilmoittaa, että tiedot ovat valmiita luettavaksi) sekä BLE_RST, joka on nollausnasta. Tämän jälkeen alustetaan Adafruit_BLE UART-objekti BTLESerial edellä olevin määrityksin.

Tämän jälkeen alustetaan muuttujat kiertovesipumppu (pinni 3) ja kastelupumppu (pinni 4), jonka jälkeen luodaan hälytysäänille array-tyyppiset muuttujat melody1 ja melody2 sekä nuottien kestoille muuttuja noteDurations.

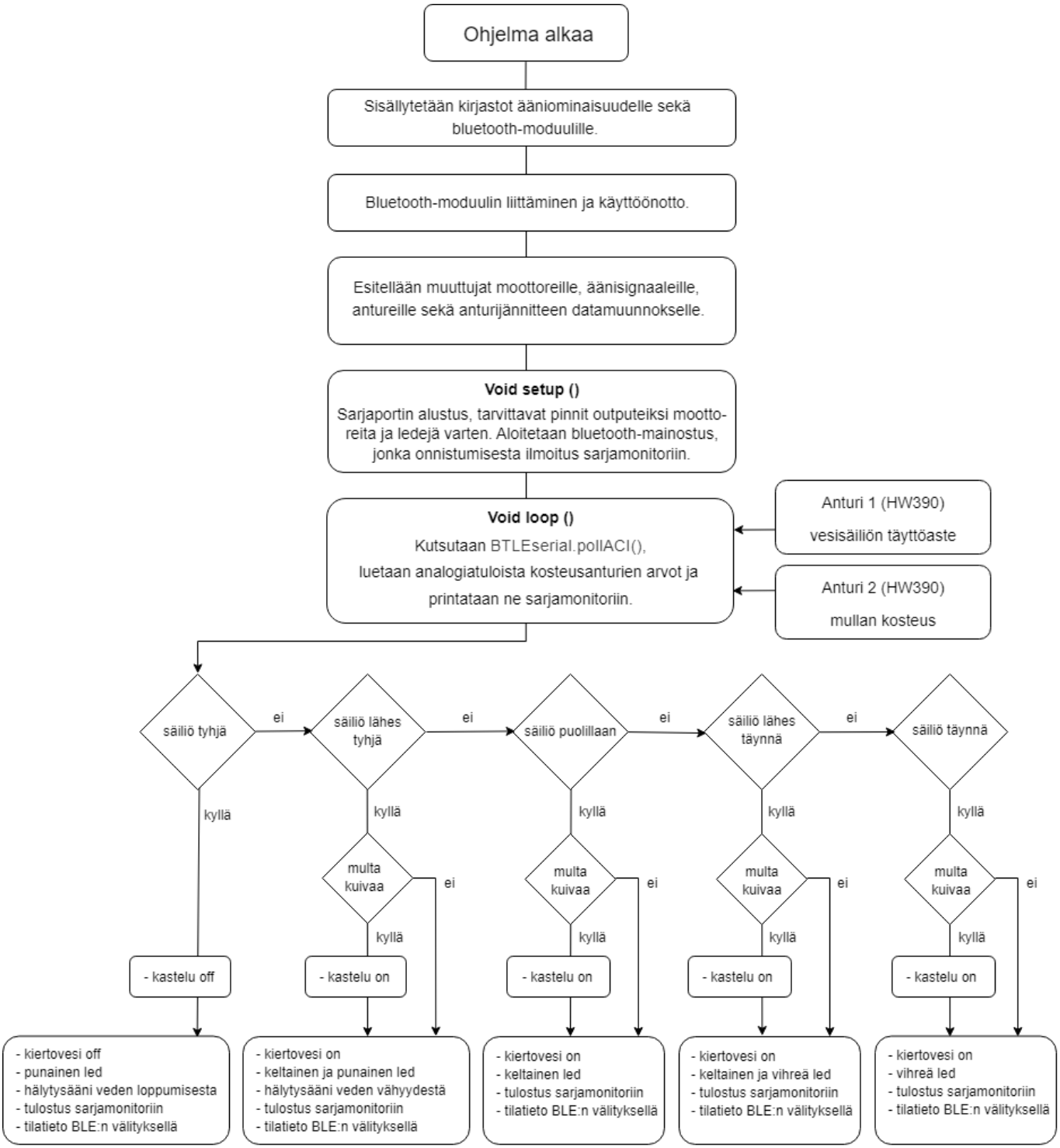
Seuraavaksi alustetaan muuttuja kosteusanturi1, joka saa Arduinon analogiatulo 0:n arvon, joka on analogatieto vesiastian täyttöastetta mittaavalta kosteusanturilta sekä kosteusanturi2, joka saa Arduinon analogiatulo 1:n arvon, joka on analogiatieto mullan kosteutta mittaavalta kosteusanturilta. Tämän jälkeen alustetaan muuttuja const int arduinon_resoluutio, joka saa arvon 1024 (koska Arduinon AD-muunnin on 10-bittinen, se voi käsitellä arvoja välillä 0-1023.

Tämän jälkeen alustetaan muuttujat arduinon käyttöjännitteelle 5V, anturin maksimi outputjänniteelle (3V) sekä jännitteiden väliselle suhteelle (lukusuhde).

Seuraavaksi ohjelmassa alkaa setup() jossa otetaan käyttöön sarjaportti, alustetaan pumppu-muuttujille määritellyt pinnit sekä pinnit 5,6 ja 7 outputeiksi, järjestelmän tilan/hälytyksen LE-Dejä varten. Seuraavaksi aloitetaan Bluetooth-laitteen mainostus.

Tämän jälkeen alkaa loop(). Rakentamani ohjelma alkaa lukemaan tietoja antureilta vasta sitten, kun se sallitaan Bluetooth-yhteydellä. Kun Bluetooth-yhteys on muodostunut, tarkistaa BTLEserial.pollACI() -kirjastofunktio aluksi, onko Bluetooth-laitteelta tullut mitään uutta. Jos on, käsittelee kirjasto tiedot nämä tiedot ja mahdollistaa ohjelman reagoinnin niihin reaaliajassa. Tämän jälkeen luetaan kosteusantureiden arvot, tehdään niille tarvittava datamuunnos ja sijoitetaan omiin muuttujiinsa, jonka jälkeen tulostetaan näiden muuttujien arvot sarjamonitoriin.

Seuraavaksi alkaa if/else if/else -ehtorakenne, joiden avulla järjestelmän tilan indikaatio ja hälytykset toteutetaan. Riippuen siitä, mikä on veden määrä vesiastiassa ja mullan kosteus (tiedot kosteusantureilta), ohjautuu suoritus oikeat ehdot täyttävään lauseeseen. Näin saadaan tieto vesiastian täyttöasteesta ja mullan kosteudesta ilmaistua sekä tarvittaessa pysäytettyä kiertovesi- ja kastelupumppu. Hälytykset saadaan osoitettua ledein ja äänimerkein; myös BLE:n välityksellä saadaan reaaliaikainen tilatieto esim. puhelimeen.



Kuva 9. Ohjelman vuokaavio.

5.6 Lopullinen ohjelmakoodi

```
#include "pitches.h" // sisällytetään äänenkorkeudet sisältävä kirjasto
#include <SPI.h> // kirjastoa käytetään kommunikoimaan SPI (Serial Peripheral Interface) -pro-
tokollaa käyttävien laitteiden kanssa
#include "Adafruit_BLE_UART.h" // kirjastoa tarvitaan BLE-kommunikaatiossa

// määritetään Arduino Uno-kortin nastat ulkoisen Adafruit nRF8001 bluetooth-moduulin käyt-
tämiseksi:

#define BLE_REQ   10
#define BLE_RDY   2
#define BLE_RST   9

// alustetaan Adafruit Bluefruit LE UART
//(Bluetooth Low Energy UART) -objekti BTLEserial:

Adafruit_BLE_UART BTLEserial = Adafruit_BLE_UART(BLE_REQ, BLE_RDY, BLE_RST);

// alustetaan muuttujat moottoreille:

int kiertovesipumppu = 4;
int kastelupumppu = 3;

// alustetaan array-muuttuja melody1 hälytysääni 1:lle (nuottien taajuudet otsikkotiedosto
pitches.h:sta):

int melody1[] = {NOTE_E4, NOTE_F4, NOTE_G4, NOTE_A5, NOTE_E4, NOTE_F4, NOTE_G4,
NOTE_A5, NOTE_E4, NOTE_F4, NOTE_G4, NOTE_A5};

// alustetaan array-muuttuja melody2 hälytysääni 2:lle (nuottien taajuudet otsikkotiedosto
pitches.h:sta):

int melody2[] = {NOTE_C4, NOTE_C6
};
```

// alustetaan array-muuttuja noteDurations (märitetään nuottien kestot): 4 = neljäsosanuotti, 8 = kahdeksasosanuotti, jne.:

```
int noteDurations[] = {32, 32, 32, 8, 32, 32, 32, 8, 32, 32, 32, 8
};

// määritetään anturi1:lle pinni A0 analogiatuloksi:
const int kosteusanturi1 = A0;

// määritetään anturi2:lle pinni A1 analogiatuloksi:
const int kosteusanturi2 = A1;

const int arduinon_resoluutio = 1024; // alustetaan muuttuja arduinon_resoluutio, joka saa arvon 1024 (koska Arduinossa on 10-bittinen AD-muunnin = 2^10=1024)

const float kayttojannite = 5.0; // alustetaan muuttuja kayttojannite, joka saa arvon 5.0 (Arduino Unon käyttöjännite)

const float max_anturin_jannite = 3.0; // alustetaan muuttuja max_anturin_jannite (maksimijännite joka saadaan anturilta)

const float anturin_lukusuhde = kayttojannite / max_anturin_jannite;
// Käyttöjännitteen ja anturin maksimijännitteen välinen suhde



//setup alkaa:

void setup() {Serial.begin(9600); // Sarjaportin alustus

pinMode(kiertovesipumppu, OUTPUT); // määritetään ko. Arduinon pinni 4 lähdöksi
pinMode(kastelupumppu, OUTPUT); // määritetään ko. Arduinon pinni 3 lähdöksi
pinMode(7, OUTPUT); // samoin
pinMode(6, OUTPUT); // samoin
pinMode(5, OUTPUT); // samoin
```

```
  if (!BTLEserial.begin()) { // aloitetaan bluetooth mainostus,

   // virheilmoitus sarjamonitoriin, mikäli ei voida aloittaa mainostusta:
Serial.println(F("Couldn't find Bluefruit, make sure it's in command mode & check wiring?"));

   while (1);
}

  // mikäli mainostus toimii, tulostus sarjamonitoriin
 Serial.println(F("Adafruit Bluefruit Command Mode Example"));

  }

//loop alkaa:

void loop() {

// BTLEserial.pollACI() -kirjastofunktio tarkistaa, onko Bluetooth-laitteelta tullut mitään uutta,
// jos on, käsittelee kirjasto nämä tiedot ja mahdollistaa ohjelman reagoinnin niihin reaaliajassa:

BTLEserial.pollACI();

int anturin1_arvo = analogRead(kosteusanturi1) * anturin_lukusuhde;
// Luetaan analoginen data kosteusanturi1:ltä ja muunnetaan se välille [0, 1023]

  int anturin2_arvo = analogRead(kosteusanturi2) * anturin_lukusuhde;
// Luetaan analoginen data kosteusanturi2:ltä ja muunnetaan se välille [0, 1023]

Serial.print("Vesisäiliön anturin arvo: ");
Serial.println(anturin1_arvo);
// Tulostetaan sarjamonitoriin anturi1:n arvo

 Serial.print("Mullan kosteusanturin arvo: ");
 Serial.println(anturin2_arvo); // Tulostetaan sarjamonitoriin anturi2:n arvo
```

```
// jos säiliö on tyhjä:
if(anturin1_arvo>956) {

  digitalWrite(kiertovesipumppu, LOW);
// kiertovesipumpun pinni LOW-tilaan (kiertovesipumppu pysähtyy)

  Serial.println("Säiliö on tyhjä < 1 litra, kiertovesipumppu seis, ei kastelua");
// Tulostetaan sarjamonitoriin

  BTLEserial.print("Säiliö tyhjä.");
// välitetään tila bluetooth slave-laitteelle (esim. puhelin)

  digitalWrite(7, HIGH);
// sytytetään LED (lähtöpinni 7:een jännite)

  delay(100);            // sekunnin viive
digitalWrite(7, LOW);    // sammutetaan LED (lähtöpinni 7:stä jännite pois)
delay(100);            // sekunnin viive
digitalWrite(kastelupumppu, LOW);
// kastelupumpun pinni LOW-tilaan (kastelupumppu pysähtyy)

// soitetaan hälytysääni1 (ajetaan soittoääni- ja niiden kesto-arrayt for-silmukassa):
for (int thisNote = 0; thisNote < 12; thisNote++) { // array-muuttujien indeksit 0-11

int noteDuration = 1000 / noteDurations[thisNote];
// lasketaan ja sijoitetaan muuttujaan noteDuration kunkin nuotin todellinen aika-arvo jaka-
malla 1000 ms noteDurations arraylle määritetyillä arvoilla

tone(8, melody1[thisNote], noteDuration);
// ajetaan kirjastofunktio tone, jolla parametreina pinni(kaiutinlähtö), nuotti(taajuus)ja nuotin
kesto)

int pauseBetweenNotes = noteDuration * 1.30;
// luodaan nuottien väliin tauko-muuttuja (noteDuration * 1.30)

   delay(pauseBetweenNotes); // ajetaan viive-funktio, parametrinaan tauko-muuttujan arvo

   noTone(8); // ajetaan noTone-funktiolla pinni 8 LOW-tilaan (ei ääntä)
```

```
  }
 }

// jos säiliö on lähes tyhjä, 3 litraa ja multa kuivaa -> kastelu alkaa:
 else if(anturin1_arvo<=956 && anturin1_arvo>=900 && anturin2_arvo>956) {

  digitalWrite(kiertovesipumppu, HIGH); // kiertovesipumppu päälle
digitalWrite(kastelupumppu, HIGH); // kastelupumppu päälle
noTone(8); // ei ääntä
  Serial.println("Säiliö lähes tyhjä, multa kuivaa, kastelu käynnissä"); // tulostus sarjamonitoriin
BTLEserial.print("Lähes tyhjä, K");
// välitetään bluetooth slave-laitteelle (esim. puhelin); K = kastelu käynnissä

digitalWrite(7, HIGH);
digitalWrite(6, HIGH);
delay(100);
digitalWrite(7, LOW);
digitalWrite(6, LOW);
delay(100);

// soitetaan hälytysääni2 (ajetaan soittoääni- ja niiden kesto-arrayt for-silmukassa):

   for (int thisNote = 0; thisNote < 2; thisNote++) { // array-muuttujien indeksit 0-1

   int noteDuration = 1000 / noteDurations[thisNote];
// lasketaan ja sijoitetaan muuttujaan noteDuration kunkin nuotin todellinen aika-arvo jaka-
malla 1000 ms noteDurations arraylle määritetyillä arvoilla

   tone(8, melody2[thisNote], noteDuration);  // ajetaan kirjastofunktio tone, jolla parametreina
pinni(kaiutinlähtö), nuotti(taajuus)ja nuotin kesto)

   int pauseBetweenNotes = noteDuration * 1.30;   // luodaan nuottien väliin tauko-muuttuja
(noteDuration * 1.30)

delay(pauseBetweenNotes);  // ajetaan viive-funktio, parametrinaan tauko-muuttujan arvo
noTone(8);  // ajetaan noTone-funktiolla pinni 8 LOW-tilaan (ei ääntä)

  }
```

```
 }

// säiliö lähes tyhjä, 3 litraa, multa kosteaa -> ei kastelua

 else if(anturin1_arvo<=956 && anturin1_arvo>=900 && anturin2_arvo<=956) {

 digitalWrite(kiertovesipumppu, HIGH); // kiertovesipumppu päälle
 digitalWrite(kastelupumppu, LOW); // kastelupumppu pois päältä
noTone(8); // ei ääntä
 Serial.println("Säiliö lähes tyhjä, multa kosteaa, ei kastelua");
 BTLEserial.print("Lähes tyhjä");
digitalWrite(7, HIGH);
digitalWrite(6, HIGH);
delay(100);
digitalWrite(7, LOW);
digitalWrite(6, LOW);
delay(100);



// soitetaan hälytysääni2, (ajetaan soittoääni- ja niiden kesto-arrayt for-silmukassa):

for (int thisNote = 0; thisNote < 2; thisNote++) { // array-muuttujien indeksit 0-1

int noteDuration = 1000 / noteDurations[thisNote];
// lasketaan ja sijoitetaan muuttujaan noteDuration kunkin nuotin todellinen aika-arvo jaka-
malla 1000 ms noteDurations arraylle määritetyillä arvoilla

tone(8, melody2[thisNote], noteDuration);
// ajetaan kirjastofunktio tone, jolla parametreina pinni(kaiutinlähtö), nuotti(taajuus)ja nuotin
kesto)

int pauseBetweenNotes = noteDuration * 1.30;
// luodaan nuottien väliin tauko-muuttuja (noteDuration * 1.30)
delay(pauseBetweenNotes);
// ajetaan viive-funktio, parametrinaan tauko-muuttujan arvo
noTone(8);  // ajetaan noTone-funktiolla pinni 8 LOW-tilaan (ääni loppuu)

  }
```

```
 }


// säiliö puolitäysi, multa kuivaa -> kastelu:

  else if(anturin1_arvo<900 && anturin1_arvo>=831  && anturin2_arvo>956 ) {

digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, HIGH);
noTone(8);

  Serial.println("Säiliö puolitäysi, 6 litraa, multa kuivaa, kastelu käynnissä");
  BTLEserial.print("Puolitäysi, K");
digitalWrite(6, HIGH);
 delay(100);
digitalWrite(6, LOW);
delay(100);

 }


 // säiliö puolitäysi, multa kosteaa -> ei kastelua:

else if(anturin1_arvo<900 && anturin1_arvo>=831  && anturin2_arvo<=956 ) {
digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, LOW);
noTone(8);
Serial.println("Säiliö puolitäysi, 6 litraa, multa kosteaa, ei kastelua");
  BTLEserial.print("Puolitäysi");
digitalWrite(6, HIGH);
 delay(100);
 digitalWrite(6, LOW);
 delay(100);

 }
```

```
// säiliö lähes täysi, multa kuivaa -> kastelu:

  else if (anturin1_arvo<831 && anturin1_arvo>=816 && anturin2_arvo>956) {
  digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, HIGH);
 noTone(8);
Serial.println("Säiliö lähes täysi, 9 litraa, multa kuivaa, kastelu käynnissä");
  BTLEserial.print("Lähes täysi, K");
 digitalWrite(6, HIGH);
  digitalWrite(5, HIGH);
delay(100);
  digitalWrite(6, LOW);
digitalWrite(5, LOW);
  delay(100);

 }



// säiliö lähes täysi, multa kosteaa -> ei kastelua:

 else if (anturin1_arvo<831 && anturin1_arvo>=816 && anturin2_arvo<=956) {
  digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, LOW);

  noTone(8);
  Serial.println("Säiliö lähes täysi, 9 litraa,  multa kosteaa, ei kastelua");
  BTLEserial.print("Lähes täysi");
  digitalWrite(6, HIGH);
 digitalWrite(5, HIGH);
 delay(100);
  digitalWrite(6, LOW);
  digitalWrite(5, LOW);
  delay(100);

 }
```

```
// säiliö täysi, multa kuivaa -> kastelu:

 else if (anturin1_arvo<816 && anturin2_arvo>956) {
 digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, HIGH);
 noTone(8);
 Serial.println("Säiliö täysi, 12 litraa, multa kuivaa, kastelu käynnissä");
 BTLEserial.print("Säiliö täysi, K");
 digitalWrite(5, HIGH);
delay(100);
digitalWrite(5, LOW);
 delay(100);

}



 // säiliö täysi, multa kosteaa -> ei kastelua:
 else {
 digitalWrite(kiertovesipumppu, HIGH);
 digitalWrite(kastelupumppu, LOW);
 noTone(8);
 Serial.println("Säiliö täysi, 12 litraa, multa kosteaa, ei kastelua");
 BTLEserial.print("Säiliö täysi");
 digitalWrite(5, HIGH);
 delay(100);
 digitalWrite(5, LOW);
 delay(100);

 }

 delay(2000); // Wait 2 second

}
```

6 Yhteenveto

Projekti oli mielenkiintoinen ja opin paljon uusia asioita antureista, moottorien ohjauksesta sekä myös Bluetooth-toiminnallisuuden rakentamisesta Arduino-ympäristöön. Lisäksi sain ratkaistua ja toteutettua periaatteellisella tasolla kastelujärjestelmän automatisoinnin niin ohjelmallisesti kuin kytkentöjenkin osalta. Projektin ulkopuolelle jäi vielä demon implementoiminen oikeaan kastelujärjestelmään sekä sen fyysinen ja ohjelmallinen säätäminen ja testaaminen tarkoituksenmukaisesti toimivaksi järjestelmäksi myös todellisessa, oikeita kasveja ja kasvualustaa sisältävässä ympäristössä.

Projekti antoi runsaasti lisätietoa ja uusia inspiraation lähteitä myös sellaisiin sovelluksiin, joissa käytettäisiin etäisyys- ja muita antureita, kiihtyvyysanturia ja moottorinohjauksia suunnanvaihdolla ja nopeudensäädöllä, vaikkapa Bluetoothin välityksellä langattomasti. Kun projektin toteutus oli ajan rajallisuuden vuoksi demotasoa, jätin myös pidemmälle viedyn toteutuksen liitäntöjen, kotelointien ja virtalähteen suhteen tekemättä. Niihin tosin on ajatusta ja alustava suunnitelmaa mielessäni – kun komponentit ja niiden kytkentäperiaate oli demoversiossa jo ratkaistu.

Kun varsinainen testattu ja toimiva prototyyppi otettaisiin käyttöön, olisi syytä tehdä kytkennät ja kokonaisuuden toteutus joka tapauksessa aikaa kestävästi ja olosuhteilta suojatusti. Liitokset tehtäisiin silloin juottaen, suunniteltaisiin ja toteutettaisiin piirilevy komponenteille ja toteutettaisiin se valmistussarjan koon mukaan tarkoituksenmukaisella tavalla. Keskusyksikölle tulisi tällöin tehdä myös asianmukainen kotelointi, jotta toteutuksesta tulee suojattu ja mietitty kiinnitystä ja käytön helppoutta ajatellen.

Mikäli tavoitteena olisi tehdä markkinoille uusi tuote ja kehitetyssä prototyypissä olisi kaupallista potentiaalia, edettäisiin luonnollisesti pitemmälle. Ensinnäkin itse tuote pitäisi suunnitella teknisesti nimenomaan teollista sarjatuotantoa ajatellen. Tuotteen valmistusprosessi, komponenttien hankintaketju, kohderyhmät ja jakelukanavat pitäisi olla hyvin mietityt ja tutkitut, jotta taloudellinen kannattavuus ja odotetut tuotot saavutettaisiin. Tuotteen tulisi täyttää lisäksi turvallisuudelle asetettavat vaatimukset ja hyväksynnät ja sen pitäisi olla käyttäjäystävällinen, pitkäikäinen, esteettisesti hyvän näköinen ja myyvä sekä tarvittaessa lisäominaisuuksilla laajennettavissa erilaisia käyttäjäryhmiä ja -tarpeita ajatellen.

7. Lähteet

1. https://docs.arduino.cc/hardware/uno-rev3/#features
2. https://arduinogetstarted.com/tutorials/arduino-soil-moisture-sensor
3. https://blog.zerokol.com/2021/01/hw-390-capacitive-soil-moisture-sensor.html
4. https://randomnerdtutorials.com/complete-guide-for-ultrasonic-sensor-hc-sr04/
5. https://lastminuteengineers.com/l298n-dc-stepper-driver-arduino-tutorial/
6. (https://learn.adafruit.com/getting-started-with-the-nrf8001-bluefruit-le-breakout)
7. https://en.wikipedia.org/wiki/Serial_Peripheral_Interface
8. (https://fi.wikipedia.org/wiki/UART)